二叉树

一、　二叉树的基本概念

1、度：一个节点的**子节点**数量

1）二叉树：有且只有n_0、n_1、n_2三种度

2）哈夫曼树：只有n_0、n_2两种度

3）重要公式推导：

$$n_0 = n_2 + 1$$

2、空指针域：一个节点的子节点或连接不存在的情况

Questions:

1）二叉树有多少个空指针域？

2）有m个结点的哈夫曼树有多少个空指针域？有多少度数为 1 的节点？

3）有 500 个节点的完全二叉树有多少个空指针域？

二、　AVL 树的建立

实战演练：

Q：输入{ 2, 1, 4, 5, 9, 3, 6, 7 }，建立 AVL 树

```cpp
// 左旋
Node* rotateLeft(Node* y) {
    Node* x = y->right;
    Node* T2 = x->left;

    x->left = y;
    y->right = T2;

    updateHeight(y);
    updateHeight(x);

    return x;
}
```

```cpp
// 右旋
Node* rotateRight(Node* y)
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    updateHeight(y);
    updateHeight(x);

    return x;
}
```

我们可以自己用程序验证一下建立的正确性：

```cpp
// AVL树节点定义
struct Node {
    int key;
    Node* left;
    Node* right;
    int height;

    Node(int k) : key(k), left(nullptr), right(nullptr), height(1) {}
};

// 获取节点高度
int height(Node* node) {
    return node ? node->height : 0;
}

// 更新节点高度
void updateHeight(Node* node) {
    if (node) {
        node->height = 1 + max(heigh
    }
}

// 计算平衡因子
int balanceFactor(Node* node) {
    return height(node->left) - heig
}

// 左旋
Node* rotateLeft(Node* y) {
```

```
Microsoft Visual Studio 调试
Level-order traversal of AVL tree:
4 2 6 1 3 5 9 7

E:\日常训练C\二叉树\x64\Debug\二叉树.exe (进程 35404)已退出，代码为 0。
按任意键关闭此窗口. . .
```

（使用层序遍历输出的）

三、二叉树中**递归**调用

我们来看几个二叉树中递归使用的例子：

Q1：交换二叉树左右节点

```
//先来看一个错误算法
TreeNode* swapLeftRight(struct TreeNode* root) {
    if (root == NULL) {
        return NULL;  // 如果节点为空，直接返回
    }
    // 递归交换左右子树，并返回修改后的根节点
    root->left = swapLeftRight(root->right);
    root->right = swapLeftRight(root->left);
}
```

分析一下错在哪里：

1、

2、

```
//正确算法
TreeNode* swap_left_and_right(TreeNode* root)
{
    if (root == NULL) return NULL;
    TreeNode* lnode = swap_left_and_right(root->left);
    TreeNode* rnode = swap_left_and_right(root->right);

    //交换当前节点
    TreeNode* t = lnode;
    lnode = rnode;
    rnode = t;

    return root;
}
```

Q2：建立二叉排序树

```
// 创建一个新节点
struct TreeNode* createNode(int value) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->value = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct TreeNode* insert(struct TreeNode* root, int value) {
    // 如果树为空，创建一个新节点并返回
    if (root == NULL) {
        return createNode(value);
    }
    // 如果值小于根节点，则插入到左子树
    if (value < root->value) {
        root->left = insert(root->left, value);
    }
    // 如果值大于根节点，则插入到右子树
    else if (value > root->value) {
        root->right = insert(root->right, value);
    }

    return root;
}
```

Q3：判断两个二叉树是否相同

```
bool isSameTree(struct TreeNode* p, struct TreeNode* q) {
    // 如果两个节点都为空，返回 true
    if (p == NULL && q == NULL) {
        return true;
    }

    // 如果其中一个节点为空，另一个节点不为空，返回 false
    if (p == NULL || q == NULL) {
        return false;
    }

    // 如果当前节点的值不同，返回 false
    if (p->val != q->val) {
        return false;
    }

    // 递归判断左子树和右子树
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}
```

Q4：求结点在二叉排序树中层次的算法

```
int level(TreeNode* root, TreeNode* key)
{
    if (root == NULL || key == NULL) return 0;
    if (key == root) return 1;
    TreeNode* queue[1000];
    int l = 0;
    int r = 0;
    int ans = 1;
    queue[r++] = root;
    while (l < r)
    {
        int times = r - l;
        for (int i = 0; i < times; i++)
        {
            if (queue[l]->left != NULL)
            {
                if (queue[l]->left == key) return ans + 1;
                queue[r++] = queue[l]->left;
            }
            if (queue[l]->right != NULL)
            {
                if (queue[l]->right == key) return ans + 1;
                queue[r++] = queue[l]->right;
            }
            l++;
        }
        ans++;
    }
}
```

KMP 算法

Q：求字符串“$aaabacaaabacaaa$”的 next 与 nextval 数组

图

一、图的一些边缘概念

1、 强联通：任意一个节点都可以经过有限步数到达其他任意节点

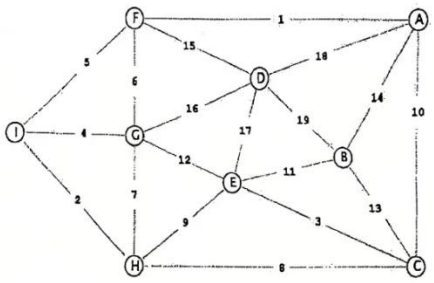
Q：含有n个顶点的强联通图至少有　条边

2、 图的度：与顶点相连边的数量

边数 = 度数/2

二、图的经典算法题目

d) **Minimum spanning trees.** Consider the following edge-weighted graph with 9 vertices and 19 edges. Note that the edge weights are distinct integers between 1 and 19.



[10 marks]

i) Complete the sequence of edges in the MST in the order that Kruskal's algorithm includes them (by specifying their edge weights).

1 ___ ___ ___ ___ ___ ___ ___ ___ **(5 marks)**

ii) Complete the sequence of edges in the MST in the order that Prim's algorithm includes them (by specifying their edge weights).

1 ___ ___ ___ ___ ___ ___ ___ ___ **(5 marks)**

排序

1、二分查找的平均查找长度

Eg. $\{1,2,3,4,5,6,7\}$

2、一次快排后的结果

Eg. $\{72,73,71,23,94,16,5\}$，$pivot=72$

3、空间复杂度

4、哈希碰撞（线性探测法与链地址法）

b) Suppose you have the following hash table, implemented using linear probing. The hash function we are using is the identity function, h(x) = x

0	1	2	3	4	5	6	7	8
9	18		12	3	14	4	21	

[12 marks]

i) In which order could the elements have been added to the hash table? **(4 marks)**

(There are several correct answers, and you should give all of them. Assume that the hash table has never been resized, and no elements have been deleted yet.)

A 9, 14, 4, 18, 12, 3, 21
B 12, 3, 14, 18, 4, 9, 21
C 12, 14, 3, 9, 4, 18, 21
D 9, 12, 14, 3, 4, 21, 18
E 12, 9, 18, 3, 14, 21,4

ii) Remove 3 from the hash table, and write down how it looks afterwards. **(4 marks)**

5、初始堆

建立$\{55,63,44,38,75,80,31,56\}$初始堆（最小堆）

时间复杂度

一、直接考察（高等数学之极限与等价问题）

非常好用的公式：

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

Algorithm Analysis

10. You are asked to select a theta notation for each of the following expressions:

- (1)$\sum_{i=1}^{n} i \lg i.$
- (2)$\sum_{m=1}^{n} \lg(m^k + c).$
- (3)$10^6 n^2 + 5 \cdot n^3 + 3 \cdot 2^n$

二、间接考察（设计满足限定时间复杂度的算法：既是难点也是突破点）

Q: Design an algorithm that takes two arrays, and returns true if the arrays are disjoint, i.e. have no elements in common.

Good: your algorithm should take $O(nlogn)$ time.

For a Very Good : your algorithm should take $O(nlogm)$ time, where n is the size of the larger array and m is the size of the smaller array, Hint for VG: since n > m, this is the same as $O(nlogm + mlogm)$.)